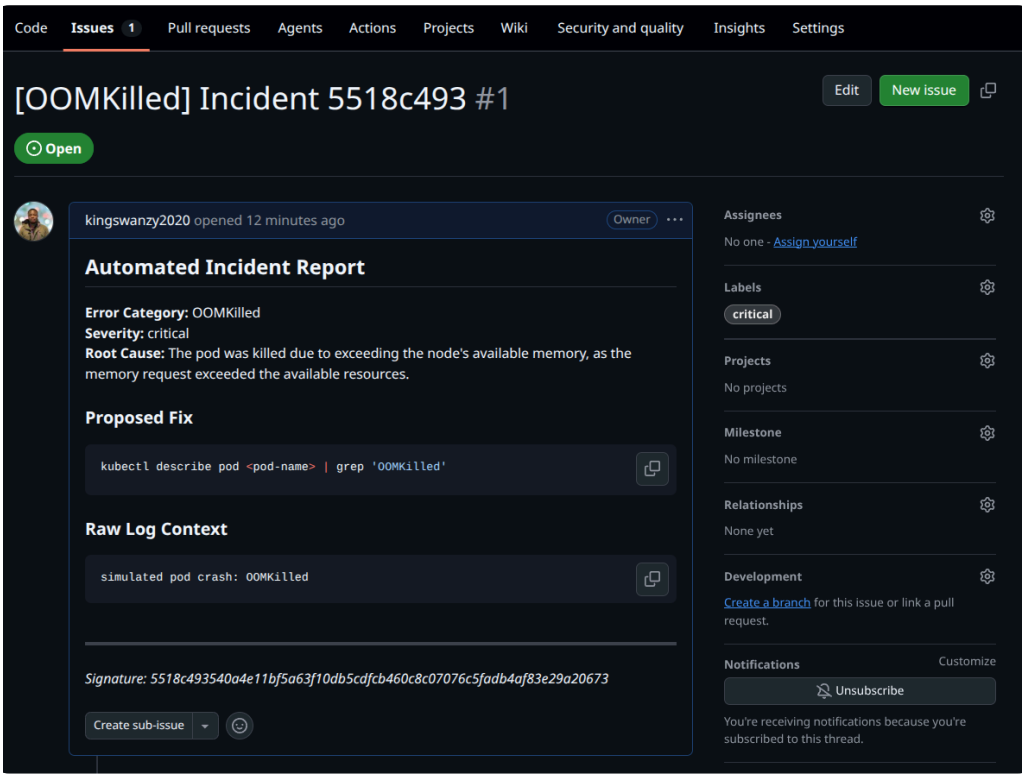
AI-POWERED LOG ANALYSIS & INCIDENT RESPONSE

By Ahmed Tetteh | May 2026



Project Vision: Autonomous Incident Response at Scale

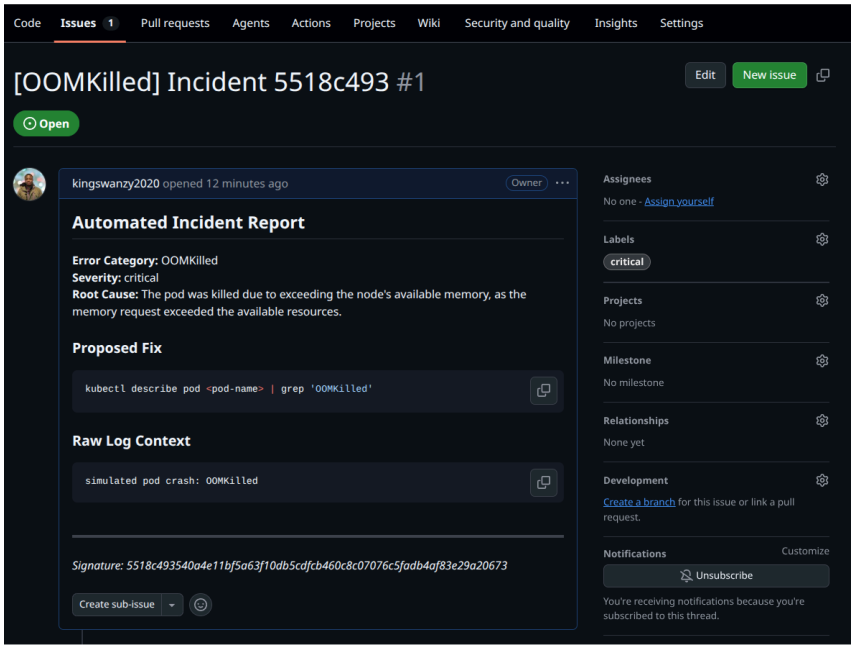
What this system does and why it matters

In this project, I'm building a complete incident response pipeline that connects Fluent Bit log collection, a FastAPI middleware service, Ollama LLM analysis, Redis caching, and automated GitHub Issue creation into a single workflow running on my local machine, so that I can remediate errors within my cluster much quicker and faster.

Proving the Full Pipeline: End-to-End Integration

Integration test goals

In this step, I'm running an end-to-end integration test, so that I can confirm if my FAST API is receiving logs from Fluent Bit running on a node within my cluster.



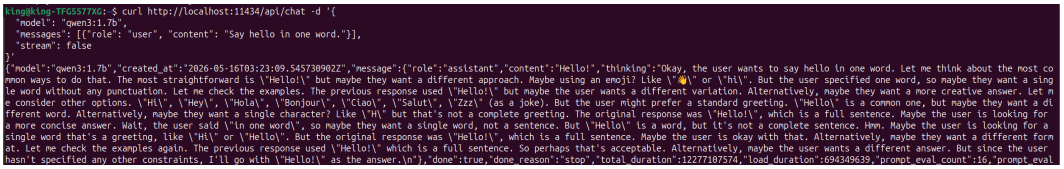
Observing the pipeline fire in real time

When the error arrived, I saw my service create a GitHub issue with a signature and cached the diagnosis in my Redis cache. Futher errors of the same signature presented as cache misses and skipped issue creation, to prevent deduplication of the same error type.

Setting Up the AI Engine with Ollama

Installing and configuring the local LLM

In this step, I'm setting up an LLM model through Ollama, so that I can it can analyze the logs generated from my cluster.



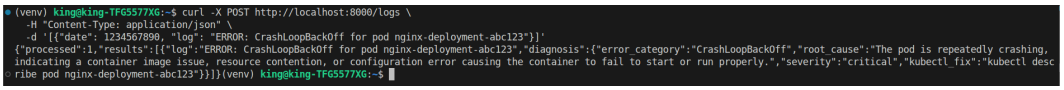
Model selection and warm-up strategy

I pulled "qwen3:1.7b" LLM model from Ollama and performed a warm-up matters because, Ollama has to first load the model into memory for subsequent responses to occur and be faster, which takes about 10 to 30 seconds.

Building the LLM Intelligence Layer

Designing AI-powered diagnosis

In this step, I'm building the LLM intelligence layer, so that my service can send out all the ERROR logs to the LLM for diagnosis.



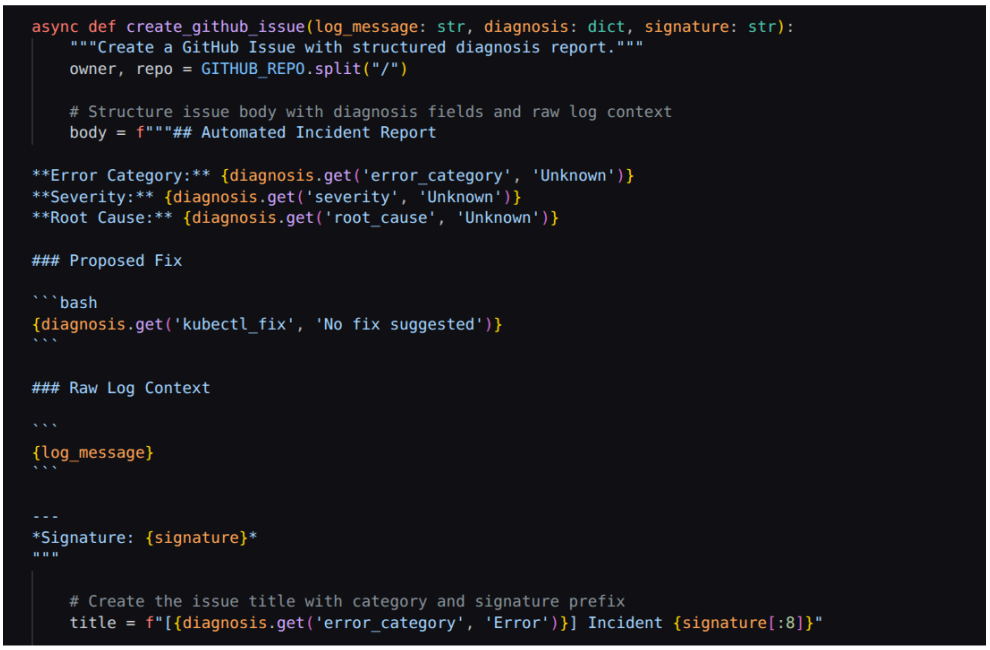
Structured output fields and rate limiting

The LLM returns error_category, root_cause, severity, and the kubectl_fix.
Rate limiting is important because we don't want to overhelm the LLM with many API calls and eat up alot of the CPU. Each inference call takes a few seconds, so if a burst of errors hits the service simultaneously, it could result in queuing of unlimited LLM calls, exhausting my machine's resources.

Automating GitHub Issue Creation

Wiring up automated incident tickets

In this step, I'm setting up the httpx integration that creates GitHub issues via the REST API, so that I can build an automated incident response piece for the SRE system.



```
async def create_github_issue(log_message: str, diagnosis: dict, signature: str):
    """Create a GitHub Issue with structured diagnosis report."""
    owner, repo = GITHUB_REPO.split("/")

    # Structure issue body with diagnosis fields and raw log context
    body = f"""## Automated Incident Report

**Error Category:** {diagnosis.get('error_category', 'Unknown')}
**Severity:** {diagnosis.get('severity', 'Unknown')}
**Root Cause:** {diagnosis.get('root_cause', 'Unknown')}

### Proposed Fix

```bash
{diagnosis.get('kubectl_fix', 'No fix suggested')}
```

Raw Log Context

```
{log_message}
```

Signature: {signature}
"""

    # Create the issue title with category and signature prefix
    title = f"[{diagnosis.get('error_category', 'Error')}] Incident {signature[:8]}"
```

Deduplication logic for clean issue tracking

My system prevents duplicates by identifying issues of the same sagnature, logs it and skips to the next thing without creating a new GitHub issue of the same signature.

Redis Caching for Resilient, Cost-Efficient Analysis

Implementing the caching layer

In this step, I'm setting up a Redis caching layer, so that repeated errors and their diagnosis can be retrieve quickly, instead of passing the same error log to the LLM.

Why caching matters under load

Caching improves reliability because it reduces the number of APi calls made directly to the LLM, improving its availability to handle other tasks.

Receiving and Filtering Kubernetes Logs with FastAPI

Building the log receiver endpoint

In this step, I'm building a FastAPI service that receives log events from Fluent Bit, filters out noise, and keeps only the errors worth investigating.

I

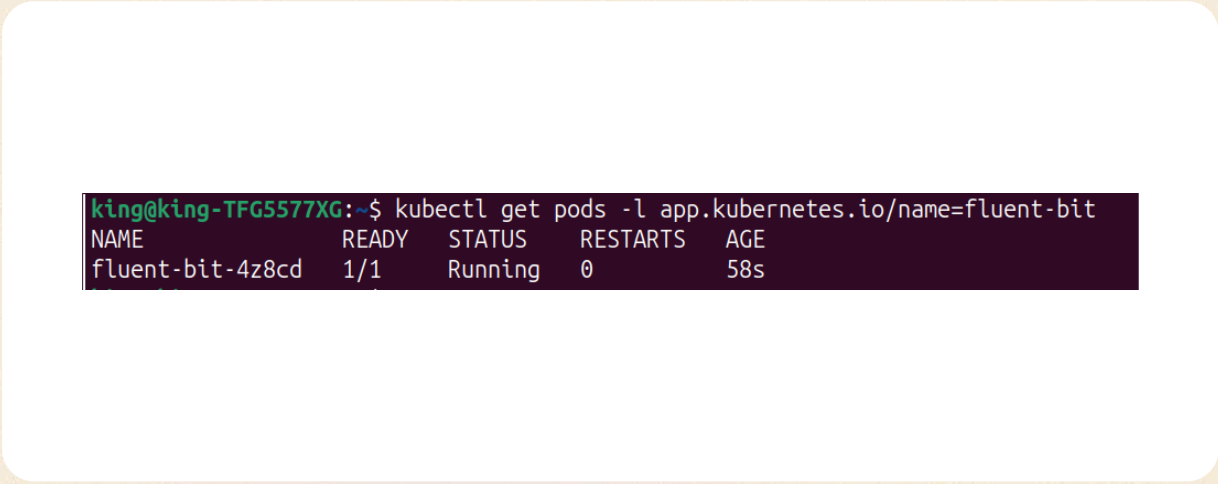
Severity filtering design decisions

The endpoint filters because not all logs contain error messages, so, we only select the logs that contain case insensitive words like "ERROR" or "CRITICAL".

Deploying Fluent Bit as a Kubernetes DaemonSet

Custom Helm configuration for log collection

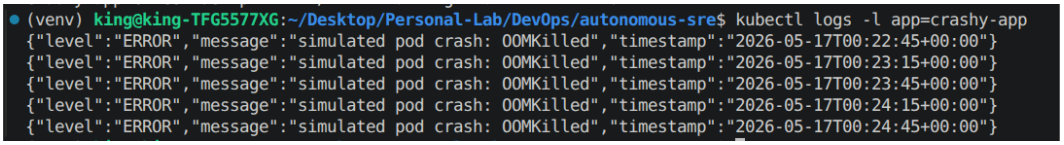
In this step, I'm deployin Fluent Bit via Helm within my cluster, so that it can send the ERROR logs generated from the containers to the REST API.



Deploying the Crashy App to Generate Real Errors

Creating a controlled error source

In this step, I'm deploying.a faulty app to my Kubernetes cluster so that I can generate real logs for the REST API.



Log format and the role of the level field

Each log line contains the level, message, and timestamp. The level field is important because when set to ERROR, Fluent Bit's grep filter will match it.

Verifying Pod-to-Host Network Connectivity

Ensuring Fluent Bit can reach the FastAPI service

In this step, I'm verifying the Pod to Host connection,. so that Fluent Bit can pass the logs to the REST API at a port on my localhost.

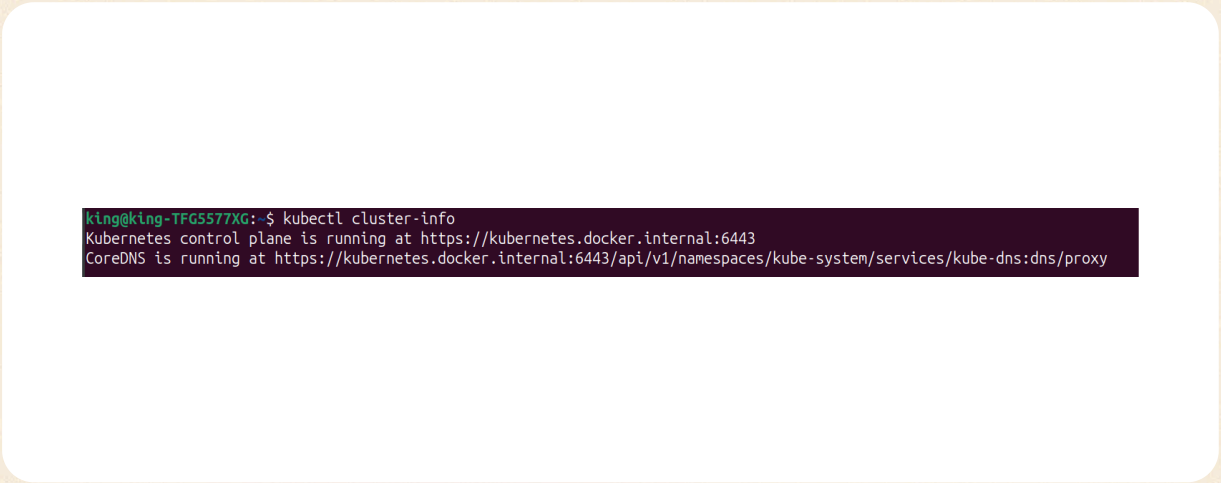
Working host address discovery

The address that worked was "host.docker.internal:8000/health" because my server was listening to traffic on all the network interfaces. I verified it by running a test curl pod within my cluster on that address for a JSON response from the health endpoint.

Configuring the Kubernetes Environment

Project environment setup

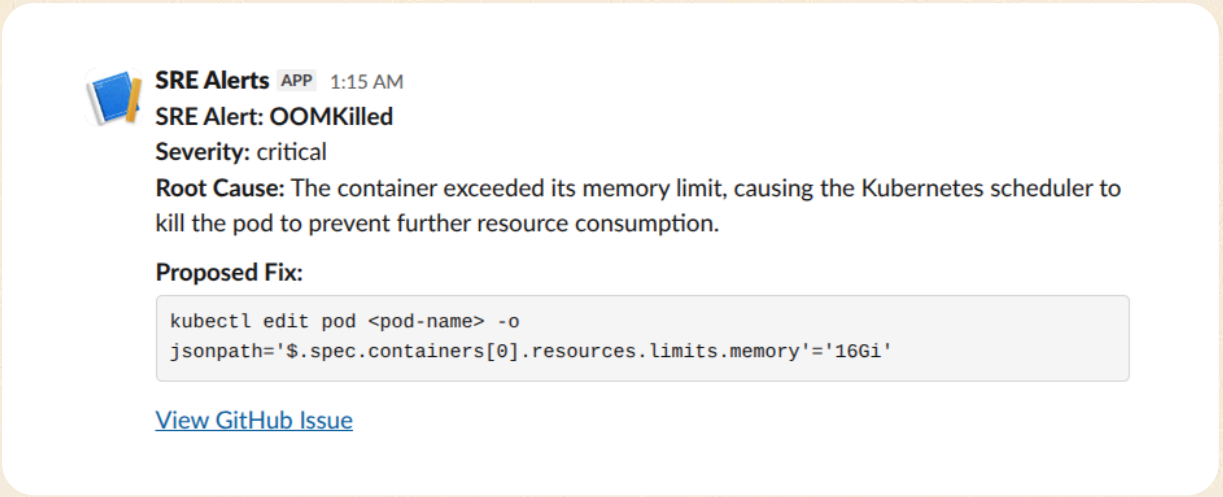
In this step, I'm setting up my project environment for SRE system.



Securing the GitHub token with least-privilege access

I granted my token Read and Write permission, scoped to a single repository because this is a security best practice that grants the token user only the needed permissions to perform the required actions within that repository.

Secret Mission: Real-Time Slack Alerting



Making Slack integration optional and safe

In this project extension, the function checks whether the Slack Webhook Url is present as an environment variable or not. If not set, it skips the function if the URL is not set, making the Slack alerts as an optional feature for the SRE system.

Reflections and Key Takeaways

Tools and concepts mastered

The key tools I used include Fast API, Kubernetes, Helm, Python, Ollama, GitHub and Slack API apps. Key concepts I learnt include how to:

1. Build an AI-powered log analysis pipeline using FastAPI, Ollama, and Redis that automatically diagnoses Kubernetes errors with a local LLM and caches results to eliminate redundant inference.
2. Deploy Fluent Bit as a Kubernetes DaemonSet with a custom pipeline that collects container logs, filters for errors, and forwards them via HTTP in real time.
3. Automate incident response by creating GitHub Issues with structured diagnosis reports, severity labels, and proposed kubectl remediation commands.

Time and challenges

This project took me approximately 8 hrs to get all the concepts and tool working together after troubleshooting. The most challenging part was setting up the /log endpoint with all the tools to parse the JSON data to them in the correct format.

What's next

I did this project today to learn how to automate and simplify incident reports, with remediation steps using AI, instead of digging through logs for errors and coming up with my own reports.